Name: Chingg Bolorkarl
email: Cbolorkar@ucsc.edu

CSE 242 Machine Learning Assignment 1

Part 1: Linear Algebra (20 points)

Q1 Given matrices:

$$A = \begin{bmatrix} 3 & 4 \\ 2 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 2 \\ 0 & -1 \end{bmatrix}$$

Compute:

$$AB,\ BA,\ (A+B)^T,\ AB^T,\ \text{trace}(A),\ \text{trace}(AB)$$

→ AB is the dot product of the i-th row of the left matrix and the j-th column of the right matrix.

$$AB = \begin{bmatrix} 3 & 4 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 0 & -1 \end{bmatrix} = \begin{bmatrix} 3(1)+4(0) & 3(2)+4(-1) \\ 2(1)+1(0) & 2(2)+1(-1) \end{bmatrix}$$

$$\therefore \boxed{AB = \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}}$$

Similarly,

$$BA = \begin{bmatrix} 1 & 2 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 3 & 4 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 1(3)+2(2) & 1(4)+2(1) \\ 0(3)-1(2) & 0(4)-1(1) \end{bmatrix}$$

$$\boxed{BA = \begin{bmatrix} 7 & 6 \\ -2 & -1 \end{bmatrix}}$$

(A+B) can be calculated by adding the corresponding elements of the two matrices that have the same dimensions.

$$A+B = \begin{bmatrix} 3 & 4 \\ 2 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 2 \\ 0 & -1 \end{bmatrix} = \begin{bmatrix} 3+1 & 4+2 \\ 2+0 & 1+(-1) \end{bmatrix}$$

$$A+B = \begin{bmatrix} 4 & 6 \\ 2 & 0 \end{bmatrix}$$

to calculate the Transpose of A+B, we swap the rows and cols of A+B.

$$\therefore \boxed{(A+B)^T = \begin{bmatrix} 4 & 2 \\ 6 & 0 \end{bmatrix}}$$

Since $B^T = \begin{bmatrix} 1 & 0 \\ 2 & -1 \end{bmatrix}$

we get, $AB^T = \begin{bmatrix} 3 & 4 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 2 & -1 \end{bmatrix}$

$$\boxed{AB^T = \begin{bmatrix} 11 & -4 \\ 4 & -1 \end{bmatrix}}$$

Trace of a matrix is the sum of the diagonal elements.

$$\boxed{\text{trace}(A) = 3 + 1 = 4}$$

Similarly,

$$\boxed{\text{trace}(AB) = 3 + 3 = 6}$$

Q2. For the following matrix:

$$M = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$$

Calculate the eigenvalues and eigenvectors. Verify your solution by showing $Mv = \lambda v$.

→

From the characteristic equation to find eigenvalues.

$$\det(M - \lambda I) = 0$$

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \qquad \lambda I = \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix}$$

$$M - \lambda I = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} - \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix} = \begin{bmatrix} 2-\lambda & -1-0 \\ -1-0 & 2-\lambda \end{bmatrix}$$

$$M - \lambda I = \begin{bmatrix} 2-\lambda & -1 \\ -1 & 2-\lambda \end{bmatrix}$$

$$\det(M - \lambda I) = (2-\lambda)^2 - 1 = 0$$

$$\therefore (2-\lambda)^2 = 1$$

$$2 - \lambda = \pm 1$$

for $2 - \lambda = 1$, $\lambda = 1$

for $2 - \lambda = -1$, $\lambda = 3$

$$\lambda_1 = 1, \quad \lambda_2 = 3$$

eigen vectors for $\lambda_1 = 1$:

we know that $Mv = \lambda_1 v$

i.e. $(M - \lambda_1 I)v = 0$

$$M - 1 \cdot I = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} - \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 2-1 & -1-0 \\ -1-0 & 2-1 \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

let $v = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$, then.

$$(M - I)v = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$$

$$= \begin{bmatrix} 1(v_1) + (-1)v_2 \\ -1(v_1) + (1)v_2 \end{bmatrix} = \begin{bmatrix} v_1 - v_2 \\ -v_1 + v_2 \end{bmatrix}$$

$$= \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$\therefore \quad v_1 - v_2 = 0 \quad \& \quad -v_1 + v_2 = 0$

multiply both sides by -1

$$-1(v_1 - v_2) = -1(0)$$

$-v_1 + v_2 = 0$

$\therefore \quad v_1 = v_2$

$\therefore$ let $v_2 = 1$ then gives $v_1 = 1$

one eigenvector is $v^{(1)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

Verification :

$$M v^{(1)} = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2(1) + (-1)(1) \\ (-1)(1) + 2(1) \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$= 1 \cdot \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \lambda_1 v^{(1)}$$

eigenvectors for $\lambda_2 = 3$.

for $(M - 3I)v = 0$:

$$M - 3I = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} - \begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix} = \begin{bmatrix} 2-3 & -1-0 \\ -1-0 & 2-3 \end{bmatrix} = \begin{bmatrix} -1 & -1 \\ -1 & -1 \end{bmatrix}$$

$$(M - 3I)v = \begin{bmatrix} -1 & -1 \\ -1 & -1 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} (-1)v_1 + (-1)v_2 \\ -1(v_1) + (-1)v_2 \end{bmatrix} = \begin{bmatrix} -v_1 - v_2 \\ -v_1 - v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\therefore \quad -v_1 - v_2 = 0$$
$$v_1 = -v_2$$

Choosing $v_2 = -1$ gives $v_1 = 1$, one eigenvector is

$$v^{(2)} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Verification :

$$Mv^{(2)} = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2(1)+(-1)(-1) \\ (-1)(1)+2(-1) \end{bmatrix} = \begin{bmatrix} 3 \\ -3 \end{bmatrix} = 3 \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$
$$= \lambda_2 v^{(2)}$$

Normalization :-

unit vectors $\frac{v}{\|v\|}$ where $\|v\| = \sqrt{v_1^2 + v_2^2}$

$$\|v^{(1)}\| = \sqrt{1^2+1^2} = \sqrt{2}, \quad \|v^{(2)}\| = \sqrt{1^2+(-1)^2} = \sqrt{2}$$

$\therefore$ normalized eigenvectors are.

$$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Q3 Prove the following property of the trace operator

$$\text{trace}(AB) = \text{trace}(BA)$$

→

Let $A \in R^{m \times n}$ and $B \in R^{n \times m}$

$$\therefore \text{trace}(AB) = \sum_{i=1}^{m} (AB)_{ii}$$

By matrix multiplication,

$$(AB)_{ii} = \sum_{j=1}^{n} A_{ij} B_{ji}$$

$$\therefore \text{trace}(AB) = \sum_{i=1}^{m} \sum_{j=1}^{n} A_{ij} B_{ji}$$

By commutative law,

$$A_{ij} B_{ji} = B_{ji} A_{ij}$$

~~But~~ Hence $$\text{trace}(AB) = \sum_{j=1}^{n} \sum_{i=1}^{m} B_{ji} A_{ij}$$

$$\sum_{i=1}^{m} B_{ji} A_{ij} = (BA)_{jj}$$

Thus, $$\boxed{\text{trace}(AB) = \sum_{j=1}^{n} (BA)_{jj} = \text{trace}(BA)}$$

Q4. Show step by step that the inverse of the matrix.

$$C = \begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix}$$ exists and compute its inverse

→

inverse of a matrix exists if it is a square matrix and its determinant is not zero.

$$\therefore \det(C) = 4(6) - 7(2) = 24 - 14 = 10 \neq 0.$$

∴ inverse of C exists.

Using $$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

we get,

$$C^{-1} = \frac{1}{10} \begin{bmatrix} 6 & -7 \\ -2 & 4 \end{bmatrix} = \begin{bmatrix} 0.6 & -0.7 \\ -0.2 & 0.4 \end{bmatrix}$$

Verification: Check $CC^{-1} = I$

$$CC^{-1} = \frac{1}{10} \begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix} \begin{bmatrix} 6 & -7 \\ -2 & 4 \end{bmatrix} = \frac{1}{10} \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix} = I$$

Part 2: Probability, MLE and MAP Estimation

Q1. Suppose a random variable X follows a discrete distribution:

$$P(X=k) = \frac{C \cdot 3^k}{k}, \quad k = 0, 1, 2, \ldots$$

(a) find the normalization constant C.

(b) Compute the expectation $E[X]$ and Variance $Var(X)$.

→

For any discrete random variable, the sum of all probabilities must equal one:

$$\sum_{k=0}^{\infty} P(X=k) = 1$$

Since, $P(X=k) = \frac{C\, 3^k}{k!}$

$$\sum_{k=0}^{\infty} C \frac{3^k}{k!} = 1$$

Take C outside summation,

$$C \sum_{k=0}^{\infty} \frac{3^k}{k!} = 1$$

exponential series formula

$$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!}$$

$x = 3$

$$\sum_{k=0}^{\infty} \frac{3^k}{k!} = e^3$$

$$\therefore ce^3 = 1 \quad \therefore \boxed{c = e^{-3}}$$

Thus,

$$P(X=k) = e^{-3}\frac{3^k}{k!}$$

Compute $E(X)$, for a discrete random variable

$$E[X] = \sum_{k=0}^{\infty} kP(X=k)$$

Substitute $P(X=k)$,

$$E[X] = \sum_{k=0}^{\infty} k\left(e^{-3}\frac{3^k}{k!}\right)$$

Since e^{-3} is constant,

$$E[x] = e^{-3} \sum_{k=0}^{\infty} k \frac{3^k}{k!}$$

the term for $k=0$ is zero,

$$E[x] = e^{-3} \sum_{k=1}^{\infty} k \frac{3^k}{3!}$$

we know that,

$$\frac{k}{k!} = \frac{k}{k(k-1)!} = \frac{1}{(k-1)!}$$

$$\therefore E[x] = e^{-3} \sum_{k=1}^{\infty} \frac{3^k}{(k-1)!}$$

Since $3^k = 3 \cdot 3^{k-1}$

we get $E[x] = 3e^{-3} \sum_{k=1}^{\infty} \frac{3^{k-1}}{(k-1)!}$

let $j = k-1$, then when $k=1$, $j=0$

$$E[x] = 3e^{-3} \sum_{j=0}^{\infty} \frac{3^j}{j!}$$

Again using $e^4 = \sum_{j=0}^{\infty} \frac{x^j}{j!}$

we have $\sum_{j=0}^{\infty} \frac{3^j}{j!} = e^3$

$\therefore E(x) = 3e^{-3} e^3$

Since $e^{-3} e^3 = e^0 = 1$

we get, $\boxed{E[x] = 3}$

also, $P(x=k) = e^{-3} \frac{3^k}{k!}$

matches the Poisson probability mass function

$P(x=k) = e^{-\lambda} \frac{\lambda^k}{k!}$

$\therefore \lambda = 3.$

for a Poisson random variable X, the mean & the variance are both equal to λ.

$\therefore \boxed{Var(x) = 3}$

Q2. Given two events A & B with $P(A) = 0.5$, $P(B) = 0.3$ & $P(A|B) = 0.6$. Compute the following (a) $P(A \cap B)$ (b) $P(B|A)$ (c) $P(A \cup B)$

→ The conditional probability of event A given that event B has occured is written as,

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

$$\therefore P(A \cap B) = P(A|B) \cdot P(B) = 0.6 \cdot (0.3) = 0.18$$

$$\therefore \boxed{P(A \cap B) = 0.18}$$

By inclusion-exclusion principle,

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

$$\therefore P(A \cup B) = 0.5 + 0.3 - 0.18$$

$$\boxed{P(A \cup B) = 0.62}$$

$$P(B|A) = \frac{P(A \cap B)}{P(A)} = \frac{0.18}{0.5} = \boxed{0.36}$$

Q3. Suppose you have a biased coin with unknown probability of heads $\theta = P(H)$

You flip the coin 5 times and observe the sequence H, H, T, H, T

(a) derive the maximum likelihood Estimate (MLE) for θ showing each step from the likelihood function.

(b) Compute the Maximum Posterior (MAP) estimate for θ assuming a prior distribution

$$P(\theta) = \begin{cases} 2\theta, & 0 \le \theta \le 1 \\ 0, & \text{otherwise} \end{cases}$$

→

the probability of getting head $= P(H) = \theta$

∴ Probability of getting tails is $P(\bar{T}) = 1 - \theta$.

observations are H, H, T, H, T.

$$L(\theta) = \theta \cdot \theta \cdot (1-\theta) \cdot \theta \cdot (1-\theta)$$

$$\therefore L(\theta) = \theta^3 (1-\theta)^2$$

The log likelihood is.

$$l(\theta) = 3 \log \theta + 2 \log (1-\theta)$$

differentiation:

$$\frac{dl}{d\theta} = \frac{3}{\theta} - \frac{2}{1-\theta}$$

by setting derivative as 0:

$$\frac{3}{\theta} = \frac{2}{1-\theta} \quad = \quad 3(1-\theta) = 2\theta$$

~~$3\theta =$~~ $\quad 3 = 5\theta \qquad \theta = \frac{3}{5}$

$$\therefore \boxed{\hat{\theta}_{MLE} = \frac{3}{5} = 0.6}$$

for MAP estimation

$$P(\theta|D) \propto P(D|\theta)\, P(\theta)$$

Substitute:

$$P(\theta|D) \propto \theta^3 (1-\theta)^2 \, (2\theta)$$

$$= 2\theta^4 (1-\theta)^2$$

Since 2 is a constant it does not affect the function of maximum.

$$\therefore P(\theta|D) \propto \theta^4 (1-\theta)^2$$

log posterior is.

$$4 \log\theta + 2 \log(1-\theta).$$

differentiate & set to 0.

$$\frac{4}{\theta} - \frac{2}{1-\theta} = 0.$$

$$\frac{4}{\theta} = \frac{2}{1-\theta} \quad \therefore 2\theta = 4(1-\theta).$$

$$\therefore 4 - 4\theta = 2\theta \quad 4 = 6\theta$$

divide both sides by 6.

$$\theta = \frac{4}{6} \quad \therefore \boxed{\hat{\theta}_{MAP} = \frac{4}{6} = \frac{2}{3} \approx 0.667}$$

Q4. Consider a gaussian distribution with parameters μ (mean) and σ^2 (variance).

Given the dataset $X = 1, 3, 5, 7$ derive the maximum likelihood estimate (MLE) for both μ & σ^2.

Show each mathematical step clearly.

→

For a Gaussian distribution the probability density function (PDF) is:

$$f(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

log likelihood is

$$l(\mu, \sigma^2) = \frac{-n}{2} \log(2\pi) - \frac{n}{2} \log\sigma^2 - \frac{1}{2\sigma^2} \sum_{i=1}^{n} (x_i - \mu)^2$$

differentiate wrt. μ:

$$\frac{\partial l}{\partial \mu} = \frac{1}{\sigma^2} \sum_{i=1}^{n} (x_i - \mu)$$

Setting to 0 gives

$$\hat{\mu}_{MLE} = \frac{1}{n} \sum_{i=1}^{n} x_i$$

for the given data.

$$\hat{\mu}_{MLE} = \frac{1+3+5+7}{4} = \boxed{4}$$

the MLE of variance.

$$\hat{\sigma}^2_{MLE} = \frac{1}{n} \sum_{i=1}^{\hat{}} (y_i - \hat{\mu})^2$$

~~$\hat{\sigma}^2_{MLE} = (1-4)^2 + (3-4)^2 + (5-4)^2 + (7-4)^2$~~

$$\hat{\sigma}^2_{MLE} = \frac{(1-4)^2 + (3-4)^2 + (5-4)^2 + (7-4)^2}{4}$$

$$= \frac{9+1+1+9}{4} = \boxed{5}$$

Q5. You observe samples drawn from a gaussian distribution with unknown mean μ and known variance $\sigma^2 = 4$. Given a gaussian prior for the mean μ

$$\mu \sim N(0,1)$$

derive the MAP estimate of μ after observing a single data point $x = 5$.

→

gaussian PDF $\quad P(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(\frac{-(x-\mu)^2}{2\sigma^2}\right)$

substitute,

$$x = 5 \;\&\; \sigma^2 = 4$$

$$P(x=5|\mu) = \frac{1}{\sqrt{8\pi}} \exp\left(-\frac{(5-\mu)^2}{8}\right)$$

prior mean $= 0$, prior variance $= 1$.

$$P(\mu) = \frac{1}{\sqrt{2\pi(1)}} \exp\left(\frac{-(\mu-0)^2}{2(1)}\right)$$

$$\therefore P(\mu) = \frac{1}{\sqrt{2\pi}} \exp\left(\frac{-\mu^2}{2}\right)$$

taking log & differentiate.

$$\log p(\mu|x) = \frac{(5-\mu)^2}{8} \frac{-\mu^2}{2} + c$$

differentiate wrt μ.

$$\therefore \frac{d}{d\mu} \log p(\mu|x) = \frac{5-\mu}{4} - \mu$$

Let the derivative equal to 0.

$$\frac{5-\mu}{4} - \mu = 0 \qquad \frac{5-\mu}{4} = \mu$$

multiply both sides by 4.

$$5 - \mu = 4\mu \qquad 5 = 5\mu$$

$$\therefore \boxed{\hat{\mu}_{MAP} = 1}$$

Q6. (Variance Expansion Proof). Prove the variance expression identity:

$$Var(X) = E(X^2) - [E(X)]^2$$

→

$$Var(X) = E[(X - E[X])^2]$$

Let $\mu = E[X]$, then,

$$Var(X) = E[(X - \mu)^2]$$

Expanding,

$$Var(X) = E[X^2] - 2\mu E[X] + E[\mu^2]$$

Since μ is constant

$$E[\mu^2] = \mu^2$$

Since $E[X] = \mu$

$$Var(X) = E[X^2] - 2\mu^2 + \mu^2$$

$$= E[X^2] - \mu^2$$

$$\therefore \boxed{Var(X) = E[X^2] - [E(X)]^2}$$

Hence, proved.

Part 3. Principal Component Analysis (Math & Programming)

Q1 Given a dataset represented by a random vector X, derive the mathematical expression for the variance of the projection onto a unit vector μ:

$$Var(\mu^T X) = \mu^T \Sigma \mu$$

document each intermediate mathematical step. and define the covariance matrix Σ.

→

Let $X \in R^d$ be random vectors representing a data point with mean vector $\mu = E[X]$.

Let $U \in R^d$ be unit vectors representing the projection direction, satisfying $\|U\|_2^2 = U^T U = 1$

projection of X onto direction U is given by:

$$y = U^T X.$$

by linearity of Expectation:

$$E[y] = E(U^T X) = U^T E(X) = U^T \mu$$

Variance is given by:

$$Var(y) = E[(y - E[y])^2]$$

substitute $y = v^T X$ & $E[y] = v^T \mu$

$$Var(v^T X) = E[(v^T X - v^T \mu)^2]$$

$$Var(v^T X) = E[v^T (X - \mu)^2]$$

$v^T (X - \mu)$ is scalar ∴ we can express it as $a^2 = a \cdot a^T$

$$\therefore (v^T (X - \mu))^2 = (v^T (X - \mu))(v^T (X - \mu))^T$$

Applying transpose rule $(AB)^T = B^T A^T$

$$(v^T (X - \mu))^T = (X - \mu)^T v$$

Substituting this in the expectation, we get.

$$Var(u^T X) = E\left[u^T (X-\mu)(X-\mu)^T u\right]$$

Because u is a deterministic unit vector, it can be factored outside.

$$Var(u^T X) = u^T E\left[(X-\mu)(X-\mu)^T\right] u$$

Covariance matrix is defined as.

$$\Sigma = E\left[(X-\mu)(X-\mu)^T\right]$$

Substituting Σ yields

$$\boxed{Var(u^T X) = u^T \Sigma u}$$

```
import numpy as np
import matplotlib.pyplot as plt
```

Q2 (5 points). Generate a synthetic 2-dimensional dataset composed of two correlated Gaussian features. Use NumPy for data generation. Visualize this dataset with scatter plots and discuss the correlation structure observed.

```
def generate_dataset(num_samples=200, random_seed=42):
    """
    Generates a synthetic 2-dimensional dataset composed of two correlated Gaussian features

    Args:
        num_samples (int): Number of observations (integer >= 2).
        random_seed (int): Seed for reproducible results.

    Returns:
        data: numpy array of shape (num_samples, 2).
    """
    if not isinstance(num_samples, (int, np.integer)):
        raise TypeError("num_samples must be an integer.")

    if num_samples < 2:
        raise ValueError("atleast 2 samples are requried.")

    # feature means
    feature_averages = np.array([5.0, 10.0])

    # non diagonal elements must be positive in covariance matrix
    # for two positively correlated variables
    covariance_matrix = np.array([
        [3.0, 2.0],
        [2.0, 2.0]
    ])

    # random number generator
    rng = np.random.default_rng(random_seed)

    # generate dataset samples drawn from gaussian distribution
    data = rng.multivariate_normal(
        mean=feature_averages,
        cov=covariance_matrix,
        size=num_samples
    )

    return data
```

```
def visualize_dataset(data):
    """
    Generate the scatter plot of the dataset.

    Args:
        data: numpy (num_samples, 2).
    """
    if data.ndim != 2 or data.shape[1] != 2:
        raise ValueError("dataset must have exactly 2 features.")

    if data.shape[0] < 2:
        raise ValueError("atleast 2 samples are required.")

    if not np.all(np.isfinite(data)):
        raise ValueError("dataset must contain only finite values.")

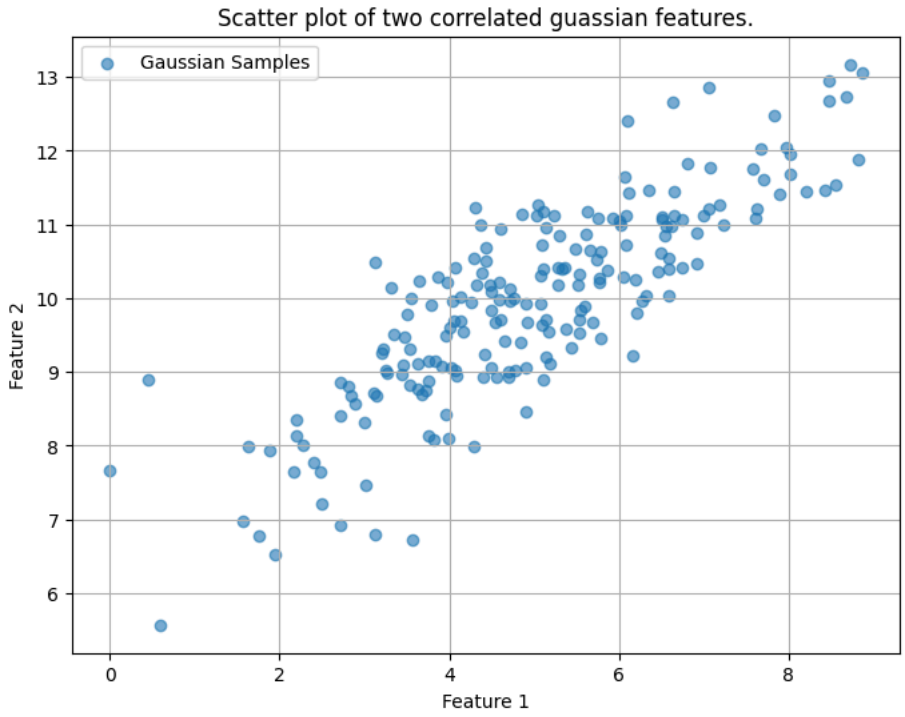
    # calculate the Karl Pearson correlation coefficient.
    corr_matrix = np.corrcoef(data, rowvar=False)
    correlation = corr_matrix[0, 1]

    # plot feature 1 against feature 2.
    plt.figure(figsize=(8, 6))
    plt.scatter(
        data[:, 0],
        data[:, 1],
        alpha=0.6,
        label="Gaussian Samples"
    )
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    plt.title("Scatter plot of two correlated guassian features.")
```

```
    plt.legend()
    plt.grid(True)
    plt.show()

    print("\nCorrelation Matrix:")
    print(corr_matrix)
    print(f"\nKarl Pearson Correlation Coefficient: {correlation:.4f}")
    print(f"\nHence, verified that the two guassian features are positively correlated.")
```

```
# generate and visualize the dataset.
# the same dataset will be used in the Q3 and Q4
data = generate_dataset(num_samples=200, random_seed=42)
visualize_dataset(data)
```




```
Correlation Matrix:
[[1.         0.83824583]
 [0.83824583 1.        ]]

Karl Pearson Correlation Coefficient: 0.8382

Hence, verified that the two guassian features are positively correlated.
```

Q3 (10 points). Implement PCA from scratch using NumPy. Follow these steps:

- Center your dataset (mean subtraction).
- Compute the covariance matrix.
- Find eigenvalues and eigenvectors of the covariance matrix.
- Select the principal components corresponding to the highest eigenvalues.

Document each step in your implementation, linking it to your mathematical derivation from Q1. Provide visualizations of your original data along with identified principal component directions.

```
def compute_pca(data, num_components=2):
    """
    Implements PCA (Principal Component Analysis).

    Inputs:
        data: 2D numpy array (samples x features).
        num_components: Number of principal components to retain.

    Returns:
        Dictionary containing centered data, feature means,
        covariance matrix, eigenvalues, eigenvectors, and projected data.
    """

    data = np.asarray(data, dtype=float)

    # validate input shape and number of samples.
    if data.ndim != 2:
```

```
        raise ValueError("input data must be a 2D array.")

    num_samples, num_features = data.shape

    if num_samples < 2 or num_features < 1:
        raise ValueError(
            "data must have at least 2 samples and 1 feature."
        )

    if not np.all(np.isfinite(data)):
        raise ValueError("data cannot contain NaN or infinity.")

    if not isinstance(num_components, (int, np.integer)):
        raise TypeError("num_components must be an int.")

    if not 1 <= num_components <= num_features:
        raise ValueError(
            "num_components must be between 1 and num_features."
        )

    # Step 1: center the dataset by subtracting its mean.
    feature_averages = np.mean(data, axis=0)
    centered_data = data - feature_averages

    # Step 2: calculate the sample covariance matrix.
    # Formula: Sigma = (X_centered.T @ X_centered) / (n - 1)
    covariance_matrix = (
        centered_data.T @ centered_data
    ) / (num_samples - 1)

    # Step 3: compute eigenvalues and eigenvectors.
    # np.linalg is more optimized for symmetric matrices.
    eigen_values, eigen_vectors = np.linalg.eigh(
        covariance_matrix
    )

    # Step 4: sort eigenvalues and eigenvectors together.
    sorted_indices = np.argsort(eigen_values)[::-1]
    eigen_values = eigen_values[sorted_indices]
    eigen_vectors = eigen_vectors[:, sorted_indices]

    # Step 5: select the top principal components based on num_components.
    principal_components = eigen_vectors[:, :num_components]

    # Step 6: project the centered observations onto the principal components.
    # @ is for matrix multiplication
    projected_data = centered_data @ principal_components

    return {
        "feature_averages": feature_averages,
        "centered_data": centered_data,
        "covariance_matrix": covariance_matrix,
        "eigen_values": eigen_values,
        "eigen_vectors": eigen_vectors,
        "principal_components": principal_components,
        "projected_data": projected_data
    }
```

```
def plot_principal_components(data, pca_results):
    """
    Visualize the 2D dataset and principal component directions.
    """
    if data.ndim != 2 or data.shape[1] != 2:
        raise ValueError("Visualization requires 2D data.")

    feature_averages = pca_results["feature_averages"]
    eigen_values = pca_results["eigen_values"]
    eigen_vectors = pca_results["eigen_vectors"]

    # scatter plot
    plt.figure(figsize=(8, 6))
    plt.scatter(
        data[:, 0], data[:, 1],
        alpha=0.5, label="Original Data"
    )

    # initiate the arrow from dataset mean.
    # the magnitude of the arrow scales with the standard deviation.
    for component_index in range(2):
        direction = eigen_vectors[:, component_index]
        arrow_length = 2 * np.sqrt(
```

```
            max(eigen_values[component_index], 0)
        )

        # matplotlib.pyplot.quiver is optimized to visualize vectors
        plt.quiver(
            feature_averages[0],
            feature_averages[1],
            direction[0] * arrow_length,
            direction[1] * arrow_length,
            angles="xy",
            scale_units="xy",
            scale=1,
            label=f"PC{component_index + 1}"
        )

    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    plt.title("Original dataset with Principal Components")
    plt.axis("equal")
    plt.legend()
    plt.grid(True)
    plt.show()
```

```
def calculate_explained_variance(eigenvalues):
    """
    Calculate individual and cumulative explained variance.

    Input:
        eigenvalues: Eigenvalues sorted in descending order.

    Returns:
        explained_variance_ratio: Variance fraction for each PC.
        cumulative_variance: Cumulative variance fraction.
    """
    eigenvalues = np.asarray(eigenvalues, dtype=float)

    if eigenvalues.ndim != 1 or eigenvalues.size == 0:
        raise ValueError("Eigenvalues must be a nonempty 1D array.")

    if not np.all(np.isfinite(eigenvalues)):
        raise ValueError("Eigenvalues must be finite.")

    if np.any(eigenvalues < -1e-10):
        raise ValueError("Covariance eigenvalues cannot be negative.")

    # Ignore tiny negative values caused by numerical rounding.
    eigenvalues = np.maximum(eigenvalues, 0.0)

    total_variance = np.sum(eigenvalues)

    # PCA cannot explain variance when every feature is constant.
    if total_variance <= 0:
        raise ValueError("Total variance must be positive.")

    explained_variance_ratio = eigenvalues / total_variance
    cumulative_variance = np.cumsum(explained_variance_ratio)

    return explained_variance_ratio, cumulative_variance
```

```
# compute PCA.
pca_results = compute_pca(data, num_components=2)

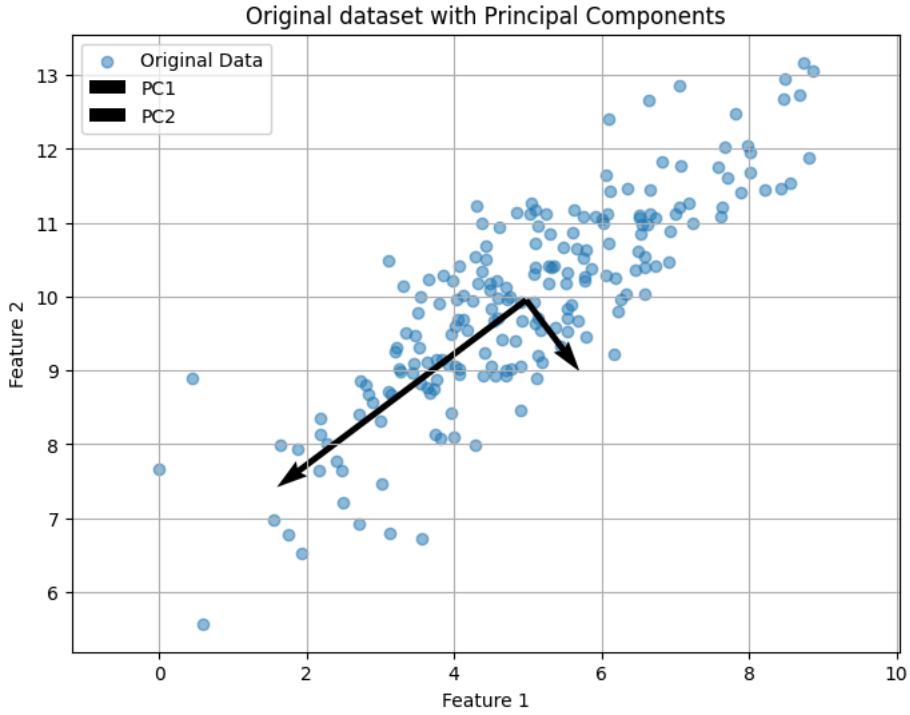
print("Eigenvalues:")
print(pca_results["eigen_values"])

print("\nEigenvectors:")
print(pca_results["eigen_vectors"])
```

```
Eigenvalues:
[4.46070748 0.36657346]

Eigenvectors:
[[-0.79971292  0.60038259]
 [-0.60038259 -0.79971292]]
```

```
# plot original observation and the principal components.
plot_principal_components(data, pca_results)
```



Q4 (10 points). Compute the proportion of total variance explained by each principal component. Discuss the results and explain the significance of dimensionality reduction. Provide insights into the trade-off between explained variance and dimensionality reduction.

```
def plot_explained_variance(
    explained_variance_ratio, cumulative_variance
):
    """Plot variance explained by each principal component."""

    num_components = len(explained_variance_ratio)
    component_numbers = np.arange(1, num_components + 1)

    # bar plot
    plt.figure(figsize=(8, 5))
    plt.bar(
        component_numbers,
        explained_variance_ratio * 100,
        label="Individual Explained Variance"
    )

    plt.plot(
        component_numbers,
        cumulative_variance * 100,
        marker="o",
        color="red",
        label="Cumulative Explained Variance"
    )

    plt.xlabel("Principal Component")
    plt.ylabel("Explained Variance (%)")
    plt.title("Explained Variance by Principal Component")
    plt.xticks(component_numbers)
    plt.ylim(0, 110)
    plt.legend()
    plt.grid(axis="y")
    plt.show()
```

```
variance_ratio, cumulative_variance = (
    calculate_explained_variance(
        pca_results["eigen_values"]
    )
)

print("\nExplained Variance:")
for index, ratio in enumerate(variance_ratio):
    print(f"PC{index + 1}: {ratio * 100:.2f}%")

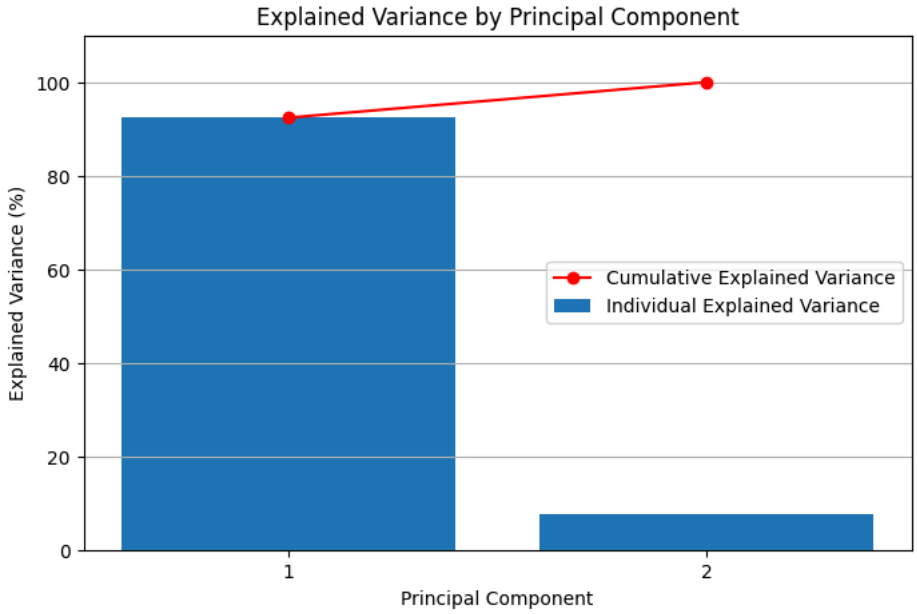
print("\nCumulative Explained Variance:")
for index, ratio in enumerate(cumulative_variance):
```

```
    print(f"PC1-PC{index + 1}: {ratio * 100:.2f}%")

plot_explained_variance(
    variance_ratio, cumulative_variance
)
```

```
Explained Variance:
PC1: 92.41%
PC2: 7.59%

Cumulative Explained Variance:
PC1-PC1: 92.41%
PC1-PC2: 100.00%
```




```
# Reduce the data from 2D to 1D using PC1.
reduced_results = compute_pca(data, num_components=1)

print("\nOriginal Data Shape:", data.shape)
print("Reduced Data Shape:",
      reduced_results["projected_data"].shape)

print(
    f"Variance retained in 1D: "
    f"{variance_ratio[0] * 100:.2f}%"
)
```

```
Original Data Shape: (200, 2)
Reduced Data Shape: (200, 1)
Variance retained in 1D: 92.41%
```

PC1 captures 92.41% of the total variance, while PC2 amounts for rest of 7.59%. This imbalance shows a positive correlation coefficient between the two features (r=0.83). Dimensionality reduction successfully eliminated the feature redundancy and orthagonalized the data. This reduction significantly reduces the memory and compute consumed for further processing.

Thus, by sacrificing a small portion of our variance we achieved a siginificant simpler data representation.